

雲原生課程 / CLOUD NATIVE SERIES

CI/CD with GitHub Actions

雲原生時代的軟體交付 : 從 commit 到 deploy 的自動化工程

講師介紹

About Us

盧柏均 (CI section) / 李佳翔 (CD section)

TSMC / NTAD

今天的目標

了解 CI/CD 的重要性, 並且可以在自己的 repo 寫出**第一條 CI/CD pipeline**。

課程大綱

今天會帶你做什麼

01

理解角色

搞懂 CI/CD 在工作流程裡的角色

02

看懂 Workflow

看懂 GitHub Actions 的 workflow 結構

03

實作 CI

在 Codespaces 跑 Fastify 專案的 CI, 用 act 本機模擬 git push event

04

容器化 + CD

把 app 容器化, 加上 CD 自動部署, 理解 branch 策略

05

認識 GitOps

雲原生時代的 CD practice

背景脈絡

先回到雲原生

過去幾堂課我們學了以下概念

Container

把 app 跟它的環境打包在一起

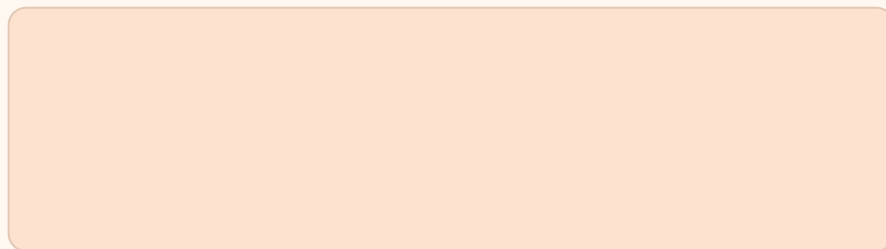
Kubernetes

大規模管理 container

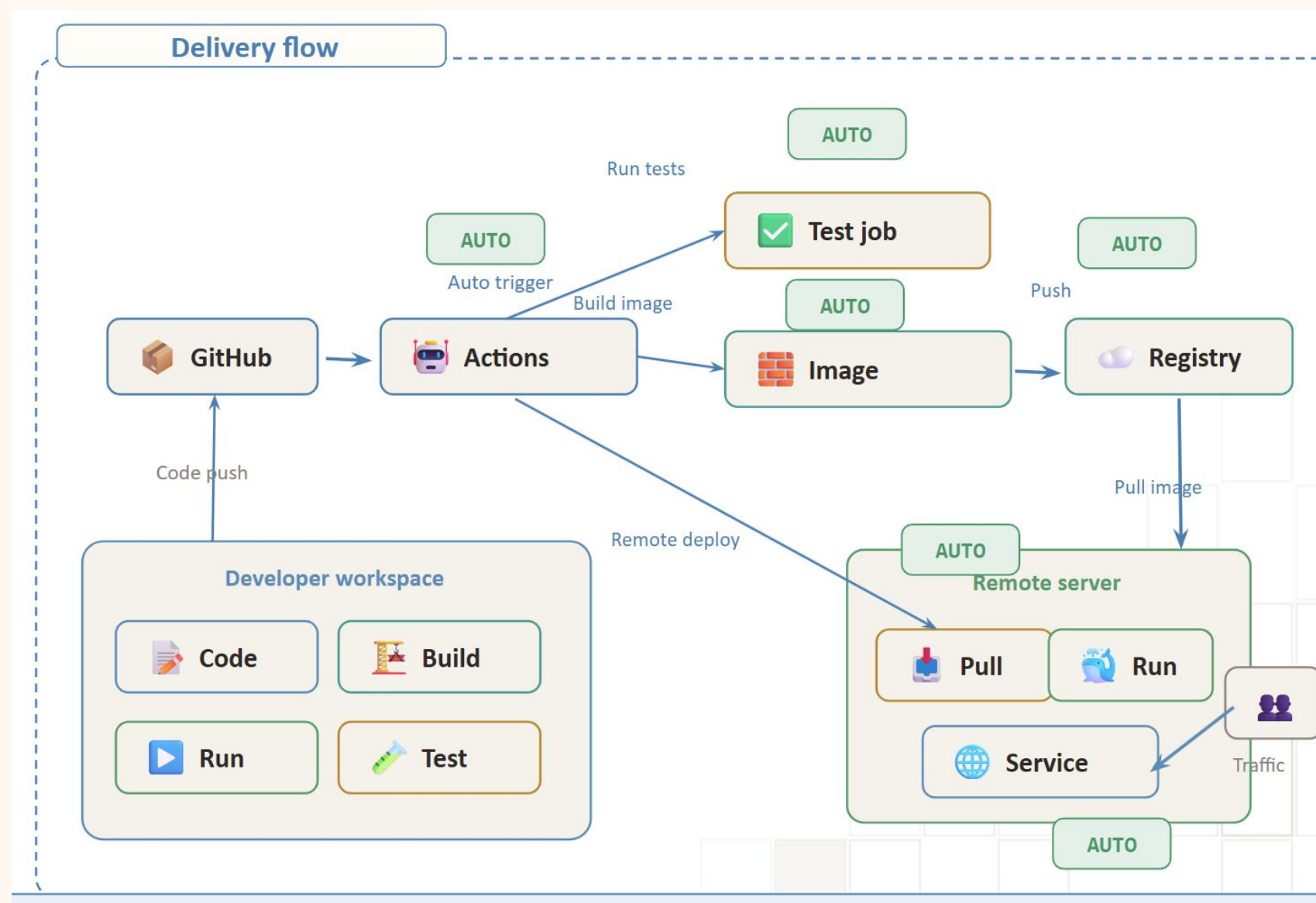
Microservices

把大系統拆成多個小服務

還缺了什麼？



回憶一下



雲原生定義

雲原生四要素

CNCF 對「雲原生」的定義包含四個技術支柱：

Containers

解決環境一致性問題

Microservices

系統可拆解、可獨立部署

Dynamic Orchestration

規模化管理

DevOps / CI/CD

讓上面三個能**持續、安全地交付**

⊗ 沒有 CI/CD, 前面三個就只是好看的架構圖

岔題

CNCF 是什麼?

詢問 ChatGPT

Cloud Native Computing Foundation (CNCF) 是一個推動「雲原生 (Cloud Native) 」技術發展的國際組織，隸屬於 The Linux Foundation 。

你可以把它理解成：

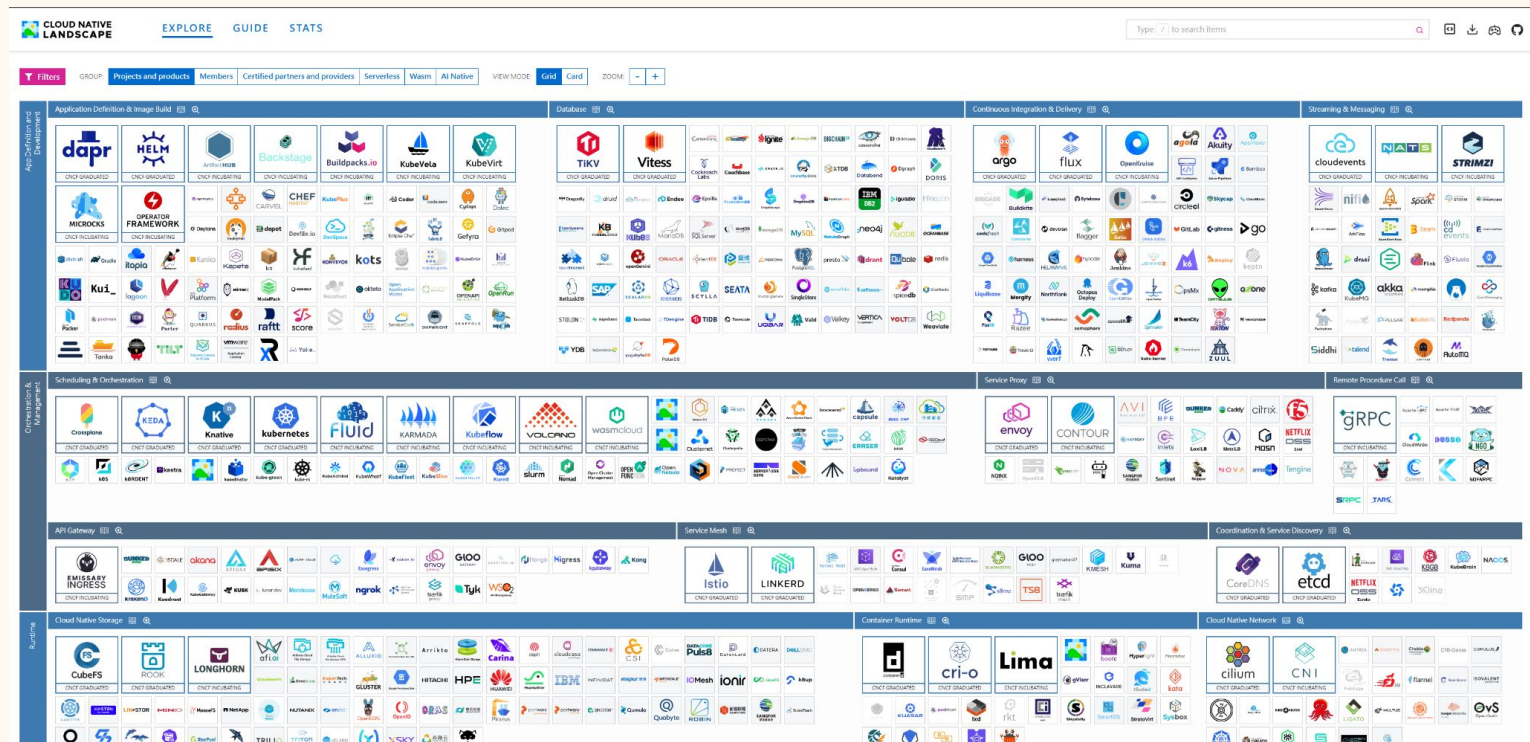
「管理與推廣 Kubernetes、生態系工具與雲端技術標準的大本營。」

最有名的專案就是：

- [Kubernetes \(K8s \)](#)
- [Prometheus](#)
- [Envoy](#)
- [Helm](#)
- [Argo CD](#)

這些很多都是 DevOps、SRE、Cloud Engineer 常接觸的技術。

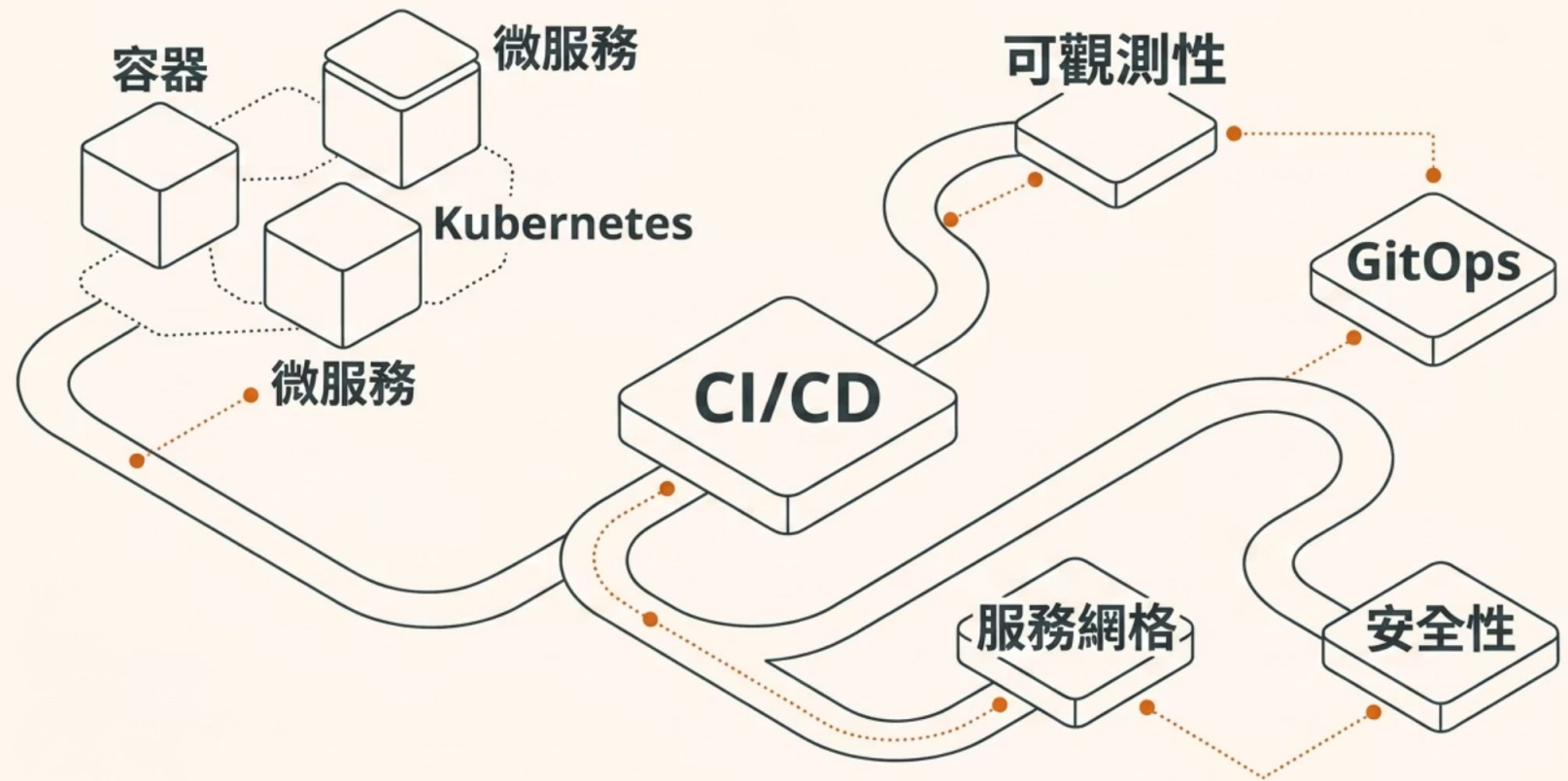
<https://landscape.cncf.io/>



課程定位

CI/CD 的定位

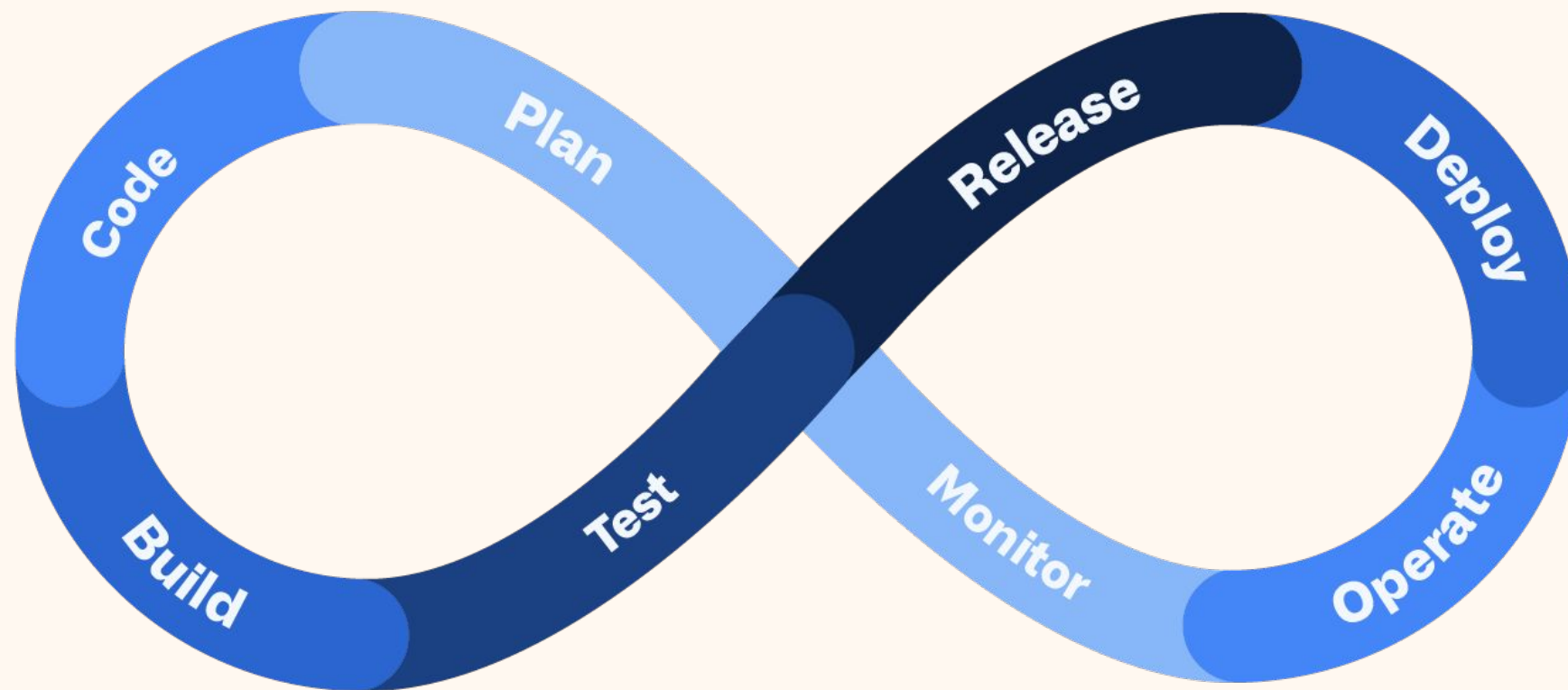
CI/CD 是把前面的雲原生概念 串成一條真正能跑的路。



軟體開發週期

任何一個軟體專案都會經歷以下階段 Analysis → Design → Development → Testing → Deployment → Maintenance。

從工程師日常視角，**每天就是下面的步驟**。沒有自動化的流程，所有階段都需要人工介入，時間根本不夠



情境

1 個人開發 1 個 或是多個專案

Analysis

Design

Development

Testing

Deployment

Maintenance



Code

Test

Build

Release

Code

Test

Build

Release

Code

Test

Build

Release

情境

多人開發1個專案

Analysis

Design

Development

Testing

Deployment

Maintenance



Code

Test

Build

Release



Code

Test

Build

Release



Code

Test

Build

Release



災難的開始:

最新版本在誰的手上?
檔案怎麼給你?
雲端硬碟上XX版?
demo 是誰的版本?

...

情境

多人開發1個專案

Analysis

Design

Development

Testing

Deployment

Maintenance



Code

Test

Build

Release



Code

Test

Build

Release



Code

Test

Build

Release

Remote
Repo



還是有問題:

誰剛剛改code? 我現在跑不起來
為什麼要動那個變數?
docker run... 欸?怎麼是舊的

...

痛點QAQ

沒有 CI/CD 的雲原生世界

想像你公司有 20 個微服務跑在 K8s 上……

工程師 A

手動 docker build → 手動 docker push → 手動 kubectl apply

工程師 B

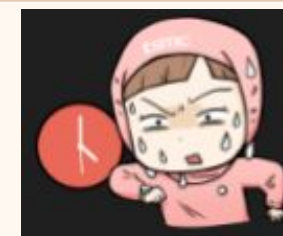
用了**不一樣版本**的 **base image** 部屬

工程師 C

在自己本地 build 的 image, **沒人知道是對應哪個 commit**

上線前一晚

20 個服務手動部署一輪, 只能保佑沒有手工出錯



核心定義

CI - Continuous Integration

Def. The practice of automating the integration of code changes from multiple contributors into a single software project

每次有人 push code, 自動執行:

- 1 抓最新的 code
- 3 Build 成可部屬的產物

- 2 執行測試
- 4 其他安全性、風格的檢查
例如: 排版是否統一、外部套件是否有漏洞

☑ 核心精神: 確保每一次的 commit 都是有品質的, 及早發現錯誤, 降低修復成本

CI Tools

市面上常見的 CI 工具：



GitHub Actions

今天要用這個



Jenkins

老牌、功能強



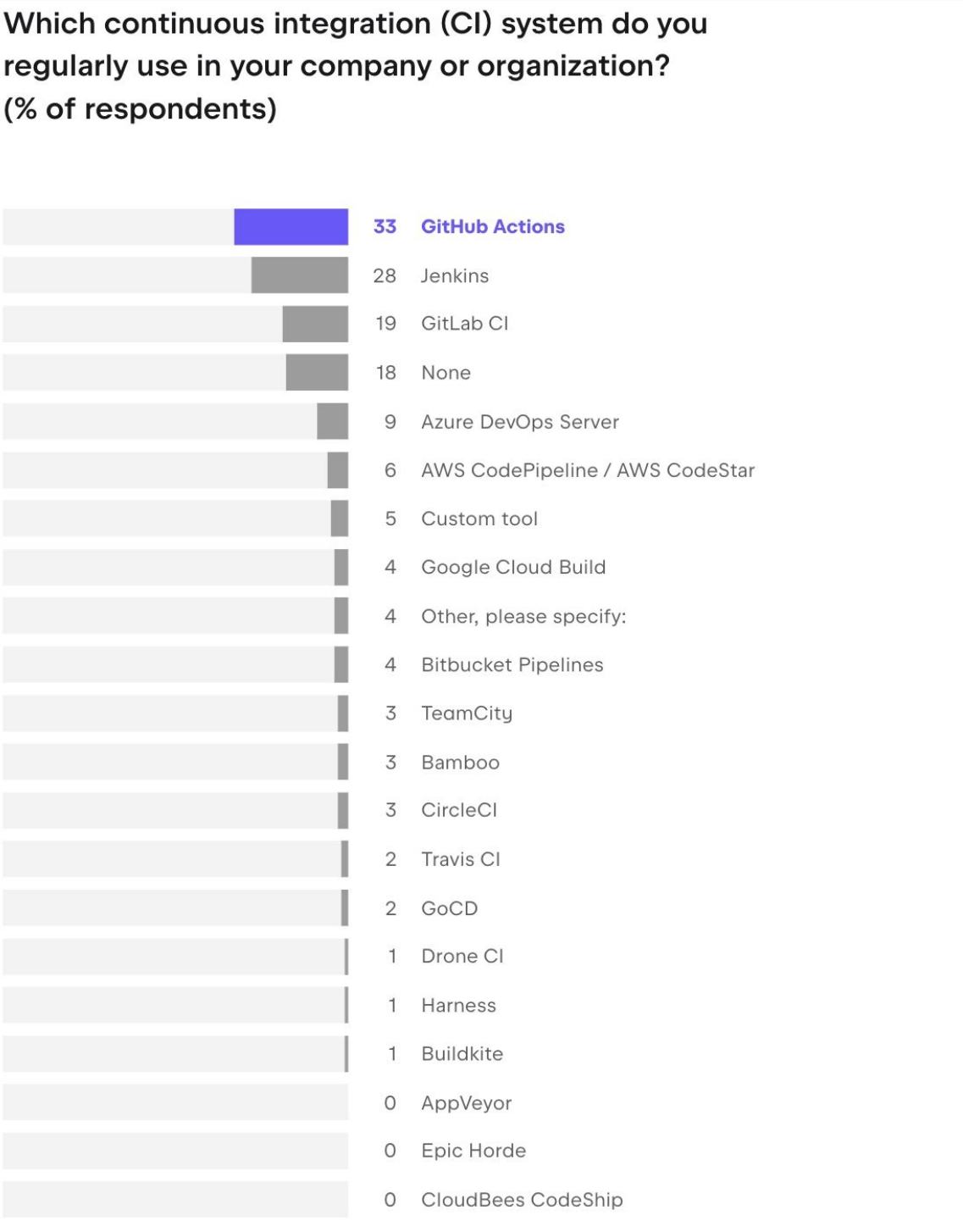
GitLab CI

跟 GitLab 綁在一起



Azure Pipelines

微軟生態



核心概念

換湯不換藥:CI Pipeline 的本質

CI 工具品牌一堆,但**核心模型都一樣** ——任何 pipeline 在描述兩件事:

Works (要做什麼)

What to do — 要做哪些步驟? (checkout、test、build...)

How to do — 怎麼做? (指令、腳本)

Resources (用什麼資源)

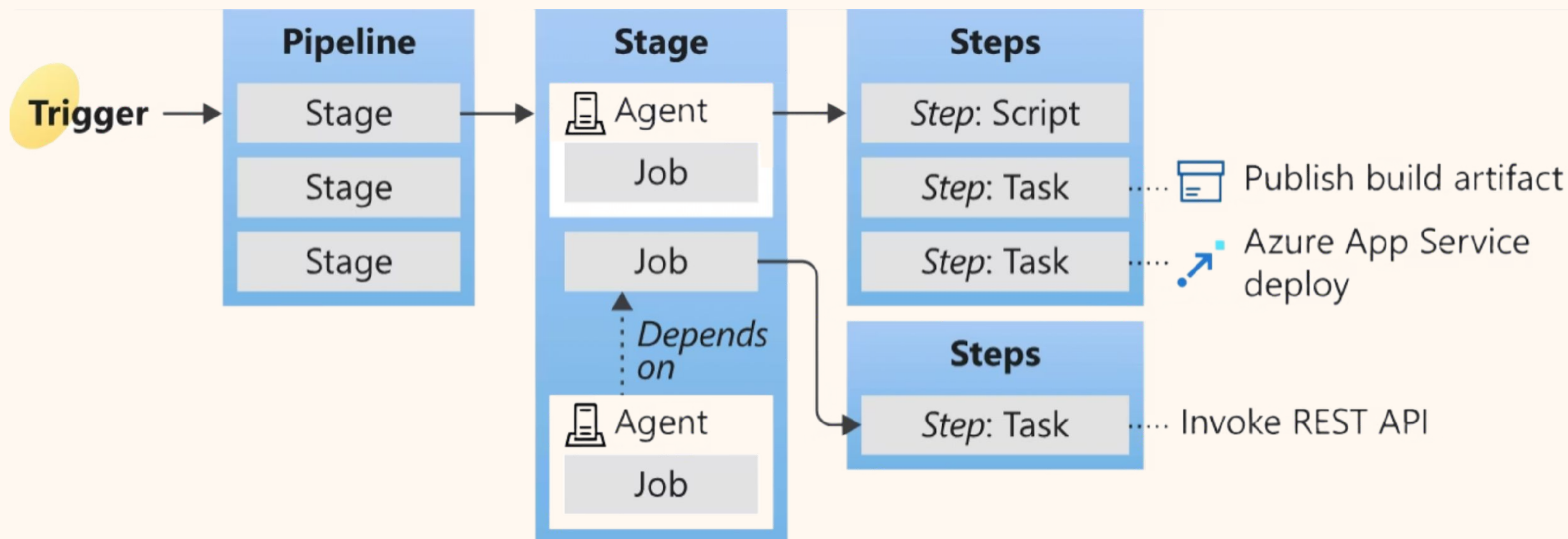
Who to do — 誰負責? (runner、agent、container)

Where to do — 在什麼環境? (node、java...)

學會這個抽象模型, **換工具只是換語法**

Azure Pipelines 的概念

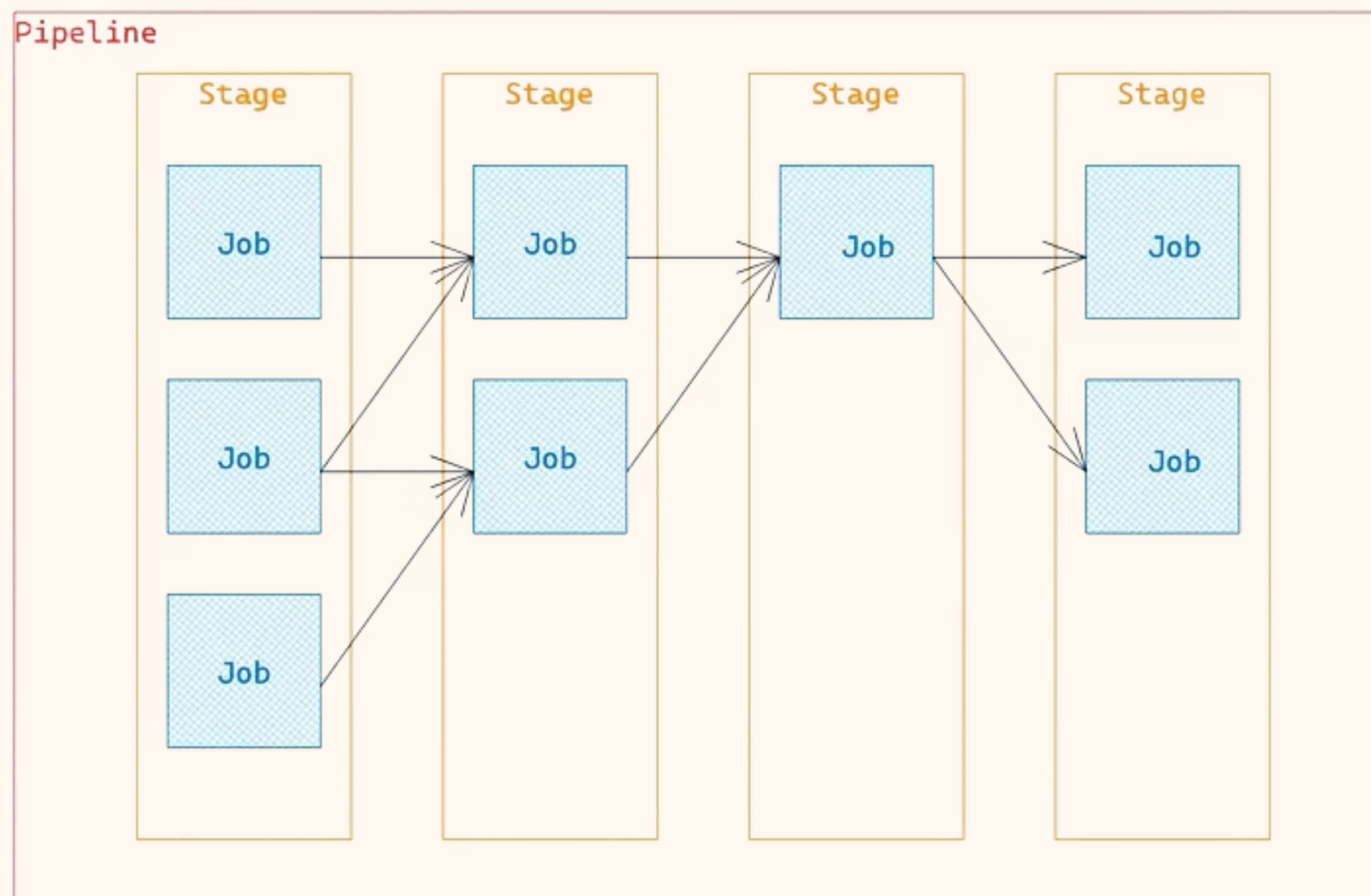
Azure Pipelines 作為 Microsoft Azure DevOps 的核心組件，提供強大的 CI/CD 能力。透過 Stage 來組織 Job。



- ❏ 每個 stage 底下還可以再有 jobs 與 steps。
通常會用 stage 來切分不同環境或流程關卡。

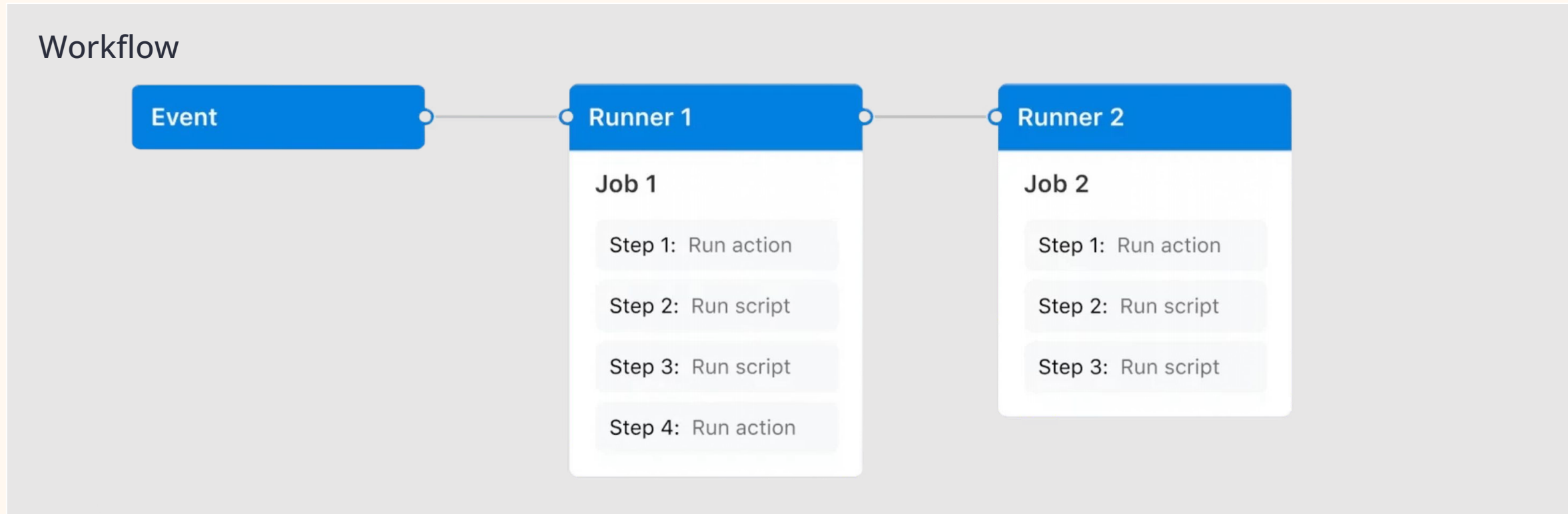
GitLab CI 的概念

與 Azure Pipelines 相似, GitLab CI 也有觸發事件、執行單元和執行環境, `Stage` 層概念來組織 Job。



你可以想像 Stage 是「橫向群組」, Job 則是「群組裡的成員」。

GitHub Actions 的概念



Event

例如程式碼推送 (push)、合併請求 (pull_request) 或定時排程 (schedule), 是觸發自動化流程的起點。



Runner

執行 Job 的虛擬機器或實體機器。GitHub 提供代管 Runner, 你也可以自架 Runner 來滿足特定需求。



Job

一個獨立的執行單元, 由一系列 Step 組成, 且總是在同一台 Runner 上執行。多個 Job 可以並行或依序執行。



Step

Job 中的最小執行步驟, 可以是執行一個指令 (run) 或使用一個別人寫好的 Action (uses)。

工具選擇

為什麼選 GitHub Actions



原生整合

跟 GitHub 原生整合, push 就跑, 零設定基礎建設



免費額度

Public repo 完全免費, 學生最友善



YAML 語法

寫過一次就會, 學習曲線低



Marketplace 生態

很多現成的 action 可用, 開箱即用

三家對照：同一件事，不同名字

不同的 CI 工具，對於概念的命名略有差異，但核心意義相似：

概念	GitHub Actions	GitLab CI	Azure Pipelines
整體	Workflow	Pipeline	Pipeline
觸發事件	Event (on:)	Trigger	Trigger
一組工作	Job	Job	Job
工作群組	(沒有)	Stage	Stage
單一步驟	Step	Script line	Step / Task
執行機器	Runner	Runner	Agent

GitHub 稱之為 **Workflow**，其他工具多稱為 Pipeline。
GitLab CI 與 Azure Pipelines 具有 **Stage** 的概念來分組工作與控制執行順序

學會 GitHub Actions，讀懂 GitLab CI / Azure Pipelines 的設定也大致能理解其核心邏輯。

為什麼 GitHub Actions 用 YAML

格式

XML

JSON

YAML

特色

標籤多，囉嗦，給機器跟人都不太友善

大括號多，給機器看的

用縮排表達結構，給人看的



縮排很重要，用空格不要用 Tab

```
<person id = "123">
  <name>
    <firstName>John</firstName>
    <lastName>Doe</lastName>
  </name>
</person>
```

```
{
  "id": "123",
  "name": {
    "firstName": "John",
    "lastName": "Doe"
  }
}
```

```
id: "123"
name:
  firstName: John
  lastName: Doe
```

YAML 在 GitHub Actions 長什麼樣

```
jobs:
  test:                # job 名字
    runs-on: ubuntu-latest # 跑在哪台機器
    steps:              # 步驟列表
      - run: npm test    # - 開頭表示 list 的一個元素
      - run: npm run lint
```

jobs: test: runs-on:
都是 key-value 結構

steps: 後面是 list
用 - 開頭, 縮排決定誰是誰的子層

✓ 看懂這個, 就看懂 GitHub Actions 80% 了

Hello World Workflow

```
name: Hello GitHub Actions      # workflow 名字
on:                             # 什麼時候要跑
  push:
    branches:
      - "*"
jobs:                           # 要做哪些事
  hello-job:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4  # uses = 用現成 action
      - name: Say Hello
        run: echo "Hello World!"    # run = 跑一行 shell
```

放在 `.github/workflows/hello.yaml`, push 到任意分支就會跑。

升級版：跑測試

```
name: Node CI
on:
  push:
    branches:
      - "**"
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 22
      - run: npm install
      - run: npm test
```

☑ 這就是一條真正能用的Node.js CI pipeline

產物概念

Artifact: CI 的產物

Build Artifact (CI 階段)

CI 跑完留下的中間產物，用於除錯或審計：

- 測試報告 (JUnit XML、coverage)
- Build 出來的 binary / dist 檔
- Log 檔

→ `actions/upload-artifact@v4`

Deployment Artifact (CD 階段)

雲原生時代，最終要部署到環境的東西：

- **Container Image** ← 今天的主角
- (傳統還有 jar / war / zip...)

→ Image 推到 Container Registry

LAB 實作

Lab 動手做

用 Codespaces + GitHub Actions + Docker + act
跑出一條完整 pipeline

Repo: <https://github.com/yubinTW/cicd-lab>

LAB 環境

Lab 環境總覽



GitHub Codespaces

雲端開發環境，瀏覽器就能寫



Node.js + Fastify + TypeScript

一個簡單的 web app + Vitest 測試框架



GitHub Actions

CI/CD 平台



Docker + act

容器化 + 本機模擬 GitHub Actions

✅ 不需要安裝任何東西，Codespaces 已經幫你裝好了 (.devcontainer 裡面定義了)

Lab 專案長什麼樣

```
cicd-lab/
├── .github/
│   └── workflows/           # ← 一開始是空的
├── snippets/
│   ├── 01_hello.yaml       # Lab-01 會用
│   ├── 02_run-test.yaml    # Lab-02 會用
│   ├── ci.yaml             # Lab-03 會用
│   └── cd.yaml             # Lab-04 會用
├── src/
│   ├── app.ts              # Fastify app (可測試)
│   └── server.ts           # 啟動 server
├── test/
│   └── app.test.ts         # Vitest 測試
├── Dockerfile
├── docker-compose.yml
└── package.json
```

這個 Lab 專案模擬一個完整的 Node.js + Fastify 服務, 包含原始碼、測試檔案和 Docker 設定。



為了讓大家循序漸進理解 GitHub Actions, 所有的 Workflow 設定檔都先放在 `snippets/` 資料夾裡。我們會分階段引導你將它們複製到 `.github/workflows/`, 這樣就能一步步感受 CI/CD Pipeline 的演進。

Step 0: Fork 並開 Codespaces

這是實作的第一步，請依照以下步驟設定您的開發環境：

前往課程儲存庫 <https://github.com/yubinTW/cicd-lab>，點擊右上角 **Fork** 按鈕以建立您的副本。

2. 進入您 Fork 出來的儲存庫。

點擊綠色的 `<>` Code 按鈕 → 選擇 **Codespaces** → 點擊 **Create codespace on main**。

4. 等待約 1-2 分鐘，讓雲端開發環境自動建立完成。

環境建立完成後，請驗證各工具的版本 (README.md)：

```
node --version
docker --version
docker compose version
act --version
```

STEP 1

Step 1: 本地跑起來

安裝依賴

```
npm ci
```

啟動服務

```
npm run build
```

```
npm run start
```

開另一個 terminal 驗證：

```
curl http://localhost:3000/
```

```
curl http://localhost:3000/health
```

✅ 應該會看到 `{"status":"ok"}` 之類的回應

STEP 2

Step 2: 跑單元測試

```
npm test
```

看到測試結果全部綠色就表示程式碼符合預期，沒有錯誤。

接著，請您故意修改 `src/app.ts` 檔案，將 `/health` 端點的回應內容改錯。再次執行 `npm test`，這時您應該會看到測試失敗（紅色）。

 這就是持續整合 (CI) 在 GitHub 上自動為您執行的任務。請務必在觀察完測試失敗後，將 `src/app.ts` 恢復原狀，否則後續推送 (push) 時，CI 將會持續失敗。

LAB 實作

Lab-01: 複製 hello.yaml + push

將位於 `snippets/01_hello.yaml` 的檔案複製到 `.github/workflows/` 目錄下，並推送到新的分支：

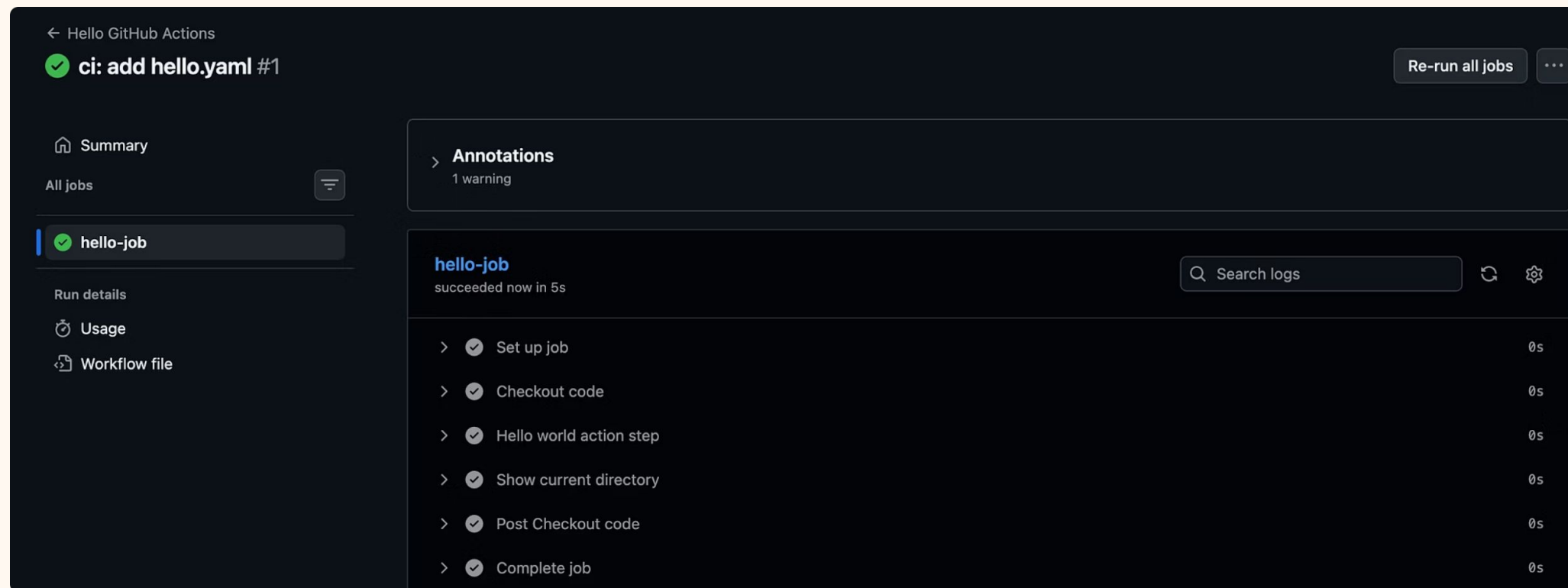
```
git checkout -b feature/ci-observe
cp snippets/01_hello.yaml .github/workflows/

git add .
git commit -m "ci: add hello.yaml"
git push origin feature/ci-observe
```

✔ 完成推送後，點擊左側導航欄的 **Actions** 分頁，您將會看到名為 **Hello GitHub Actions** 的 Workflow 已經自動開始運行。

觀察:Hello GitHub Actions 在做什麼

點進 **Hello GitHub Actions** workflow run, 並點開每個 step 檢視 log:



01

Checkout code

GitHub Actions runner 會將您的程式碼儲存庫複製(clone) 到執行環境中。

02

Hello world action step

執行一個簡單的 `echo "Hello World!"` 命令, 輸出至 log。

03

Show current directory

執行 `ls -al` 命令, 列出 checkout 後的檔案結構, 驗證程式碼已存在。



請記住, 每個 Runner 都是一台**全新的 VM 或 Container**, 它在執行您的 Workflow 之前是空的。只有經過 `Checkout code` 步驟後, 您的程式碼才會被複製到這個環境中, 供後續步驟使用。

Lab-02: 複製 run-test.yaml + push

接下來，請將位於 `snippets/02_run-test.yaml` 的檔案複製到 `.github/workflows/` 目錄下，並推送到您目前的分支：

```
cp snippets/02_run-test.yaml .github/workflows/  
  
git add .  
git commit -m "ci: add run-test.yaml"  
git push origin feature/ci-observe
```

完成推送後，點擊左側導航欄的 **Actions** 分頁，您將會看到以下兩個 Workflow 自動開始運行：

Hello GitHub Actions 

Run Tests 

 ci: add run-test.yaml	<code>feature/ci-observe</code>	 now
Hello GitHub Actions #2: Commit <code>269c97f</code> pushed by <code>sychen6192</code>		 7s
 ci: add run-test.yaml	<code>feature/ci-observe</code>	 now
Run Tests #1: Commit <code>269c97f</code> pushed by <code>sychen6192</code>		 In progress

觀察:Run Tests 的 Artifact

Artifacts			
Produced during runtime			
Name	Size	Digest	
 test-report	424 Bytes	sha256:abdef2e963473a9c20d81e1948364869fbd...	  

01

進入 Workflow 執行頁面

點進 **Run Tests** Workflow 的執行頁面。

03

尋找 Artifacts 區塊

滑動頁面到**最底部**，您會看到一個名為**Artifacts** 的區塊。

02

檢視測試結果

點開 `Run tests` 步驟，仔細查看 Vitest 執行測試的輸出日誌。

04

下載測試報告

在此區塊中，您會發現一個名為`test-report` 的壓縮檔，下載並解壓後，內容是 `vitest-junit.xml` 格式的測試報告。

 這就是我們在課程中提到的**Build Artifact** —— CI Pipeline 執行後所產生的**中間產物**。在真實專案中，除了測試報告，還會包含程式碼覆蓋率報告、編譯後的二進位檔案或打包好的應用程式等，這些都是後續CD 部署的重要依據。

CI 怎麼跟 PR 結合

確保不會把壞掉的 code merge 進主分支：



 可以在 GitHub repo 設定：**CI 沒過就不准 merge**

CI → CD

從 CI 走到 CD (Continuous Delivery)

1

CI

CODE · BUILD · TEST

驗證並打包 code

2

Release

通過驗證的產物 (Artifacts)

3

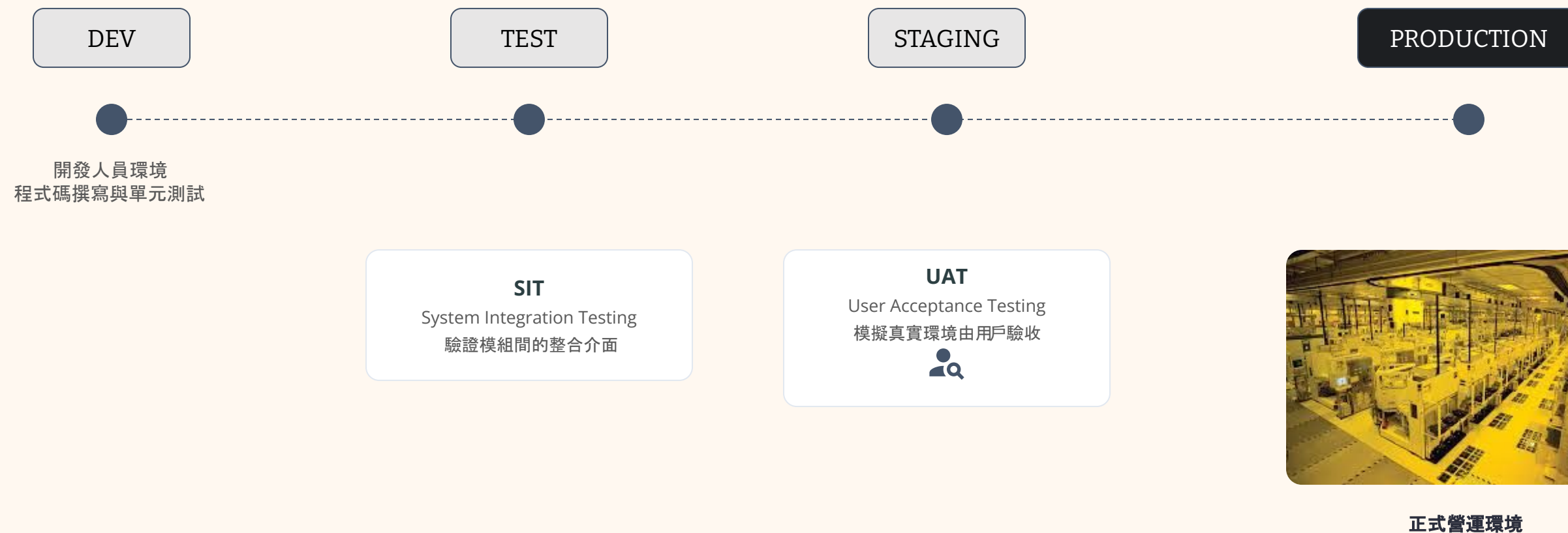
CD

DEPLOY · OPERATE · MONITOR

送到實際環境跑

CI 把 code 驗證好、打包好 → CD 把這個產物送到實際環境跑。

軟體生命週期環境



良好的環境隔離是確保系統穩定性的關鍵，從開發到正式上線需經過嚴謹的自動化與人工測試流程。

兩個 CD 的差異

Delivery vs Deployment

Continuous Delivery

自動部署到測試、staging 的環境，**但要人工核准** 才會上 production

Continuous

Deployment

從測試到 production **完全自動部署**，不需要人為介入



企業口語講 "CD" 通常指前者 (Continuous Delivery)

部署細節

Deploy 時要考慮什麼

部署不只是「丟個檔案上去」, 要考慮:



Artifact

要部署哪個版本?



Replica

開幾個 instance?

如果都用手動部署、維運 會發生.....



Architecture

一台 VM? K8s? Serverless?

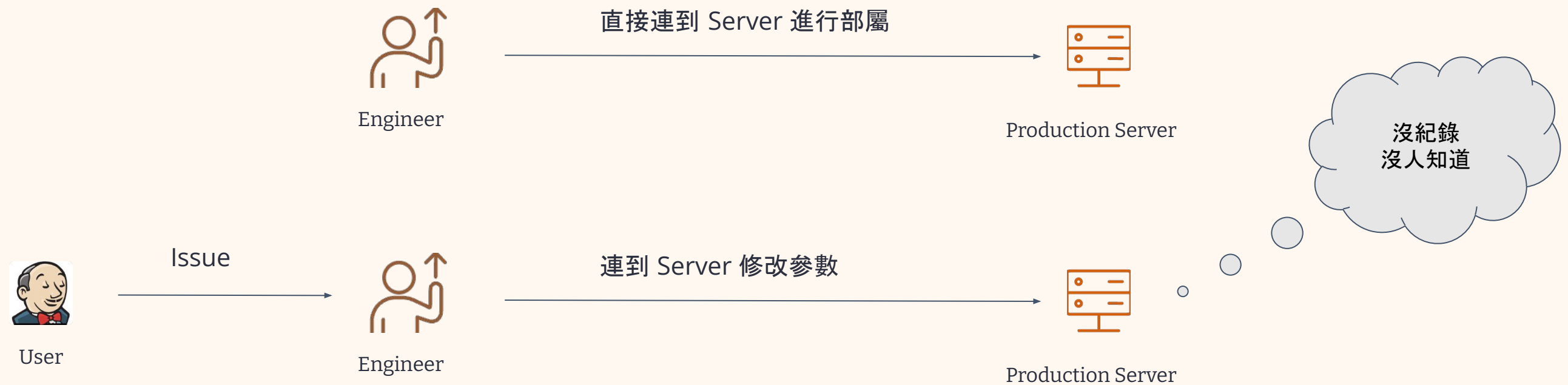


Secrets 管理

DB 連線、API key — 不能寫死在 code

Git 的重要性

如果都用手動部屬、維運會發生.....





這個參數是做什麼用的?
為什麼是這個值?
誰改的?
改了會有什麼影響?
我要怎麼知道改動的結果?

通靈中...

系統參數設定紀錄 (有人知道這個是幹麻的嗎?)

- [PROD] timeout: 30 (別動, 改了會掛)
- [PROD] retry: 3 (之前改成5好像爆過)
- [STG] feature_x: true (AE 說要開)
- [PROD] cache_ttl: 600 (前同事說這樣比較穩)
- [PROD] connection_pool: 20 (不知道為什麼是20)
- [ALL] log_level: info (debug會噁爆磁碟)
- [PROD] api_rate_limit: 100 (被客訴過太嚴格)
- [STG] mock_payment: false (測試時要記得關)
- ...還有很多沒人記得的設定...

部屬流程 (口耳相傳版)

1. 先連到那台跳板機
2. 切到 /opt/legacy-app 目錄
3. vi config.yaml (小心編排)
4. 改完記得 systemctl restart
5. 看一下 log 有沒有噴錯
6. 沒事的話跟群組說一聲

* 詳細細節請自行通靈 *

上次誰改的?
忘了...

新人報到
第一天

這台是
正式環境!!

活著
就好

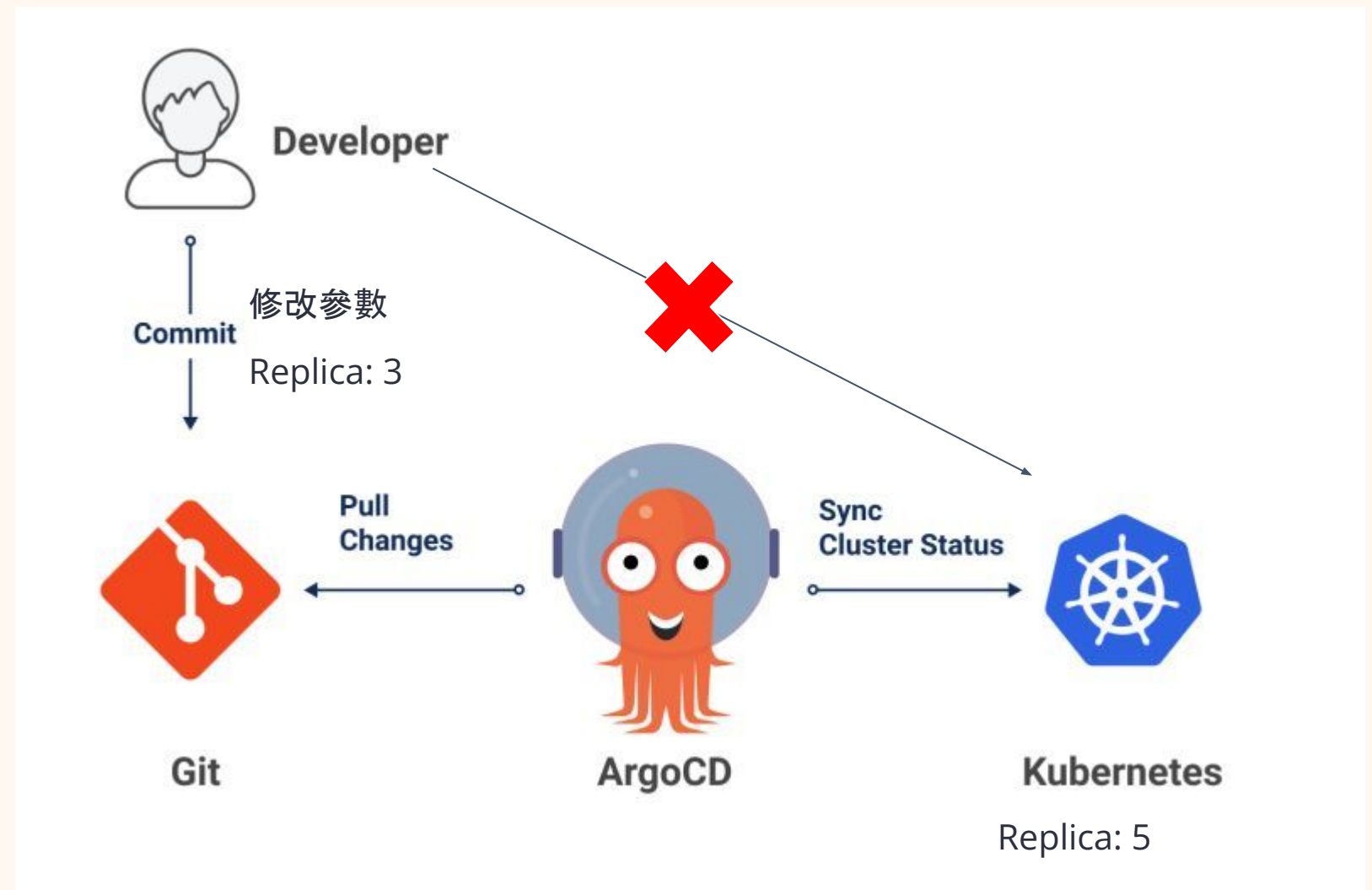
GitOps

透過 Git 留存修改紀錄，並自動化部屬，減少人工維運犯錯的機會

ArgoCD

業界最紅的 GitOps CD 工具：

- Single Source of Truth
- Tracable — 版本控管可追溯、方便 Rollback
- Declarative — 你描述，它去達成
- 自動 sync Git → K8s cluster

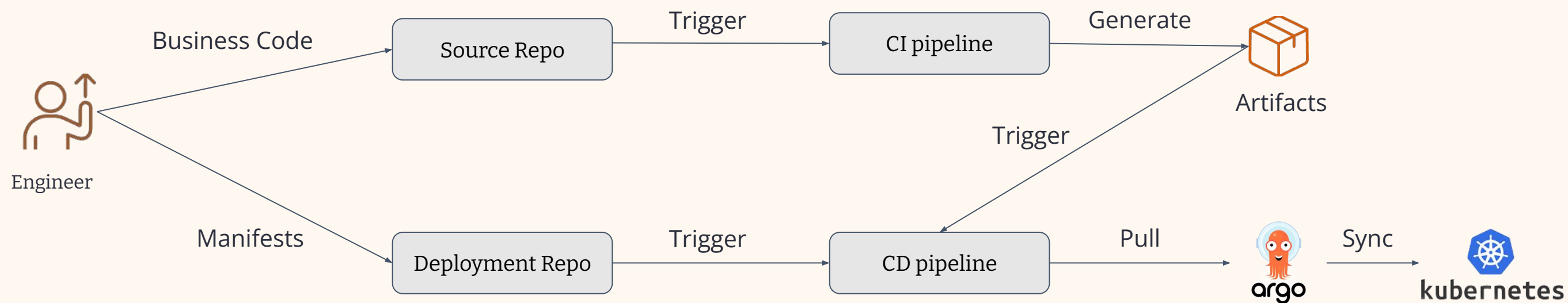


Source / Deploy 職責分離

Source repo: 應用 code (CI 跑這裡)

Deployment repo: K8s manifest / Helm (ArgoCD watch 這裡)

好處: 對於工程師, 只要改 **Code** 即可, 部署、掃描等工作全部自動化



CI/CD 的價值

CI/CD 帶來的世界 🌈

對團隊

- Bad Quality 在 commit 後快速抓到, 不是 release 那天
- Release 變日常, 不是大工程

對工程師個人

- 專心寫 code, 不用手動跑流程
- Reviewer 看 PR 可以專心在設計、邏輯

接下來, 我們動手做 🖱️

act 是什麼

`act` 是一個強大的工具，能讓您在本機端執行 GitHub Actions workflow，對於除錯和快速迭代 CI/CD 流程非常有幫助。

	act push	git push
在哪跑	本機 / Codespace	GitHub 雲端
用途	除錯、快速迭代	真實 CI/CD 紀錄
速度	5 秒到位	排隊 + 拉環境

 詳細資訊可參閱：[官網](#) 或 [GitHub 儲存庫](#)

Lab-03: 用 act 模擬 push

act 工具不僅能模擬完整的 CI/CD 流程，還能模擬特定的事件類型（例如 push），讓您在本地端就能測試 Workflow，大幅提升開發效率。

您可以直接在分支上執行 `act push` 模擬推送事件：

```
act push
```

若想只針對單一 Workflow 進行測試，可以加上 `-W` 參數：

```
act push -W .github/workflows/01_hello.yaml  
act push -W .github/workflows/02_run-test.yaml
```

進階用法：若不想切換分支，但希望模擬特定分支的推送事件，可以使用 `--env GITHUB_REF` 參數：

```
act push --env GITHUB_REF=refs/heads/your-feature-branch
```

 第一次執行 `act` 會自動拉取必要的 Docker 映像檔（例如 `catthehacker/ubuntu`），這可能需要約 1-2 分鐘。為了節省時間，您可以在課程開始前先執行 `docker pull catthehacker/ubuntu:act-latest` 預先拉取映像檔。

進階 WORKFLOW

接下來：進階版的 ci.yml + cd.yml

目前為止，`hello.yml` 和 `run-test.yaml` 已經讓您對 GitHub Actions 有了基本認識。

但業界真實的 CI/CD pipeline 會比這兩個檔案執行更多複雜的任務。

接下來，我們將帶您解析 `snippets/ci.yml` 和 `snippets/cd.yml` 這兩個進階設定檔，了解它們如何處理更複雜的 CI/CD 流程。

Lab-04 的目標

在實際應用中，我們希望透過自動化流程達成以下目標：

1 每次提交都跑 CI

確保每一次程式碼提交都能觸發 CI 流程，自動驗證程式碼品質

2 每次提交都建構 Image

每一次提交都建構 Image，確保可以接續自動部屬的任務

3 Image Tag 區分版本

利用 Image Tag 明確區分開發中的 Feature 版本與正式發佈的 Release 版本，確保版本可追溯，便於後續的錯誤排 查。

4 只從 Release 分支部署

嚴格限制只有來自 Release 分支的變更才能觸發正式環境部署，避免非預期的部署造成環境不穩定。

接下來，我們將深入解析 `snippets/ci.yaml` 與 `snippets/cd.yaml`，看看它們是如何實現這些進階目標的。

Branch 策略

「我哪個 branch 該跑 CI？哪個該跑 CD？」

Branch	跑什麼	Image Tag
feature/*	跑 CI、可跑 CD	sha-a1b2c3d
main	跑 CI	—
release/*	跑 CI + CD	release-1.4.0

 每個 image 都能對應到一個明確的 commit / 版本，這是雲原生可追溯性的基礎。

ci.yml — 觸發條件與權限

讓我們打開 `snippets/ci.yml`, 從頭開始解析這個進階的CI Workflow 設定。

```
name: ci

on:
  push:
    branches: [ '**' ]    # 所有 branch 都跑
  pull_request:          # PR 也跑

permissions:
  contents: read          # 最小權限原則
```

觸發條件 (on)

這個 Workflow 會在每次程式碼被 `push` 到任何分支時自動觸發。同時，當有新的 `pull_request` 被建立或更新時，也會觸發CI 流程，確保每次合併前程式碼都能通過測試。

權限設定 (permissions)

這裡採用「最小權限原則」，僅賦予Workflow 讀取儲存庫內容 (`contents: read`) 的權限。這是一個重要的安全實踐，避免因權限過大而造成潛在的資安風險。

CI.YML 解析

ci.yml — 兩個進階配置

Concurrency: 重複 **push** 自動取消舊的

```
concurrency:  
  group: ci-${{ github.workflow }}-${{ github.ref }}  
  cancel-in-progress: true
```

同一 branch 連續 push, 只跑最新那次, 省 runner 時間

Matrix: 多版本並行驗證

```
strategy:  
  fail-fast: false  
  matrix:  
    node-version: ['22', '24']
```

同時用 Node 22 跟 24 各跑一次, 驗證升級風險

ci.yml — 主要 Steps

```
- name: Checkout
  uses: actions/checkout@v4          # 抓 code

- name: Setup Node.js
  uses: actions/setup-node@v4        # 裝 Node
  with:
    node-version: ${{ matrix.node-version }}
    cache: npm

- name: Install dependencies
  run: npm ci                        # 裝依賴

# add your steps here, e.g. lint, test, build etc.
# - run: npm run typecheck          # 型別檢查
```

⊗ 每一行都是一個 quality gate, **任何一步失敗整個 CI 失敗**

ci.yml — 計算 Image Tag

```
- name: Compute image tag
  id: meta
  run: |
    BRANCH_NAME="${GITHUB_HEAD_REF:-${GITHUB_REF#refs/heads/}}"
    if [[ "$BRANCH_NAME" == release/* ]]; then
      VERSION="${BRANCH_NAME#release/}"
      IMAGE_TAG="release-${VERSION}"
    else
      IMAGE_TAG="sha-${GITHUB_SHA:0:7}"
    fi
```

Feature Branch

sha-a1b2c3d

Release Branch

release-1.4.0



每個 image 都能對應到一個明確的commit / 版本。

CI.YML 解析

ci.yml — Build Image + 觸發 CD

```
- name: Build Docker image
  if: ${ matrix.node-version == '24' }
  run: docker build -t my-app:${ steps.meta.outputs.image_tag } .

# 另一個 job
cd:
  if: ${ startsWith(github.ref, 'refs/heads/release/') }
  needs: ci
  uses: ../.github/workflows/cd.yml
```

 關鍵: 只有 release/* branch、CI 成功, 才會觸發 CD

cd.yml — 看一下結構

```
on:
  workflow_dispatch: # 可手動觸發
  workflow_call: # 可被 ci.yml 呼叫

jobs:
  deploy:
    if: ${ startsWith(github.ref_name, 'release/') }
    steps:
      - name: Checkout
        uses: actions/checkout@v4
      - name: Compute release image tag
        ...

      - name: Build and deploy with Docker Compose
        run: docker compose up -d --build
      - name: Verify health endpoint
        run: curl -fsS http://localhost:3000/health
```

Lab-04:複製 ci/cd + act 模擬

首先, 將進階版的 `ci.yaml` 和 `cd.yaml` 複製到您的 workflows 目錄:

```
cp snippets/ci.yaml .github/workflows/  
cp snippets/cd.yaml .github/workflows/
```

Feature Branch 觀察 ci.yaml:

在功能分支上模擬 push 事件, 觀察 CI 流程的行為:

```
git checkout -b feature/a  
act push  
docker images    # 查看 build 出來的 image
```

 您會觀察到 Docker Image 的 Tag 會是類似 `sha-a1b2c3d` 的格式, 這代表了本次commit的 SHA 值。

Release Branch 觀察 ci.yaml + cd.yaml:

切換到發布分支, 模擬 push 事件, 觀察 CI/CD 流程的協同工作:

```
git checkout -b release/1.0.0  
act push  
docker images    # 查看 release tag image
```

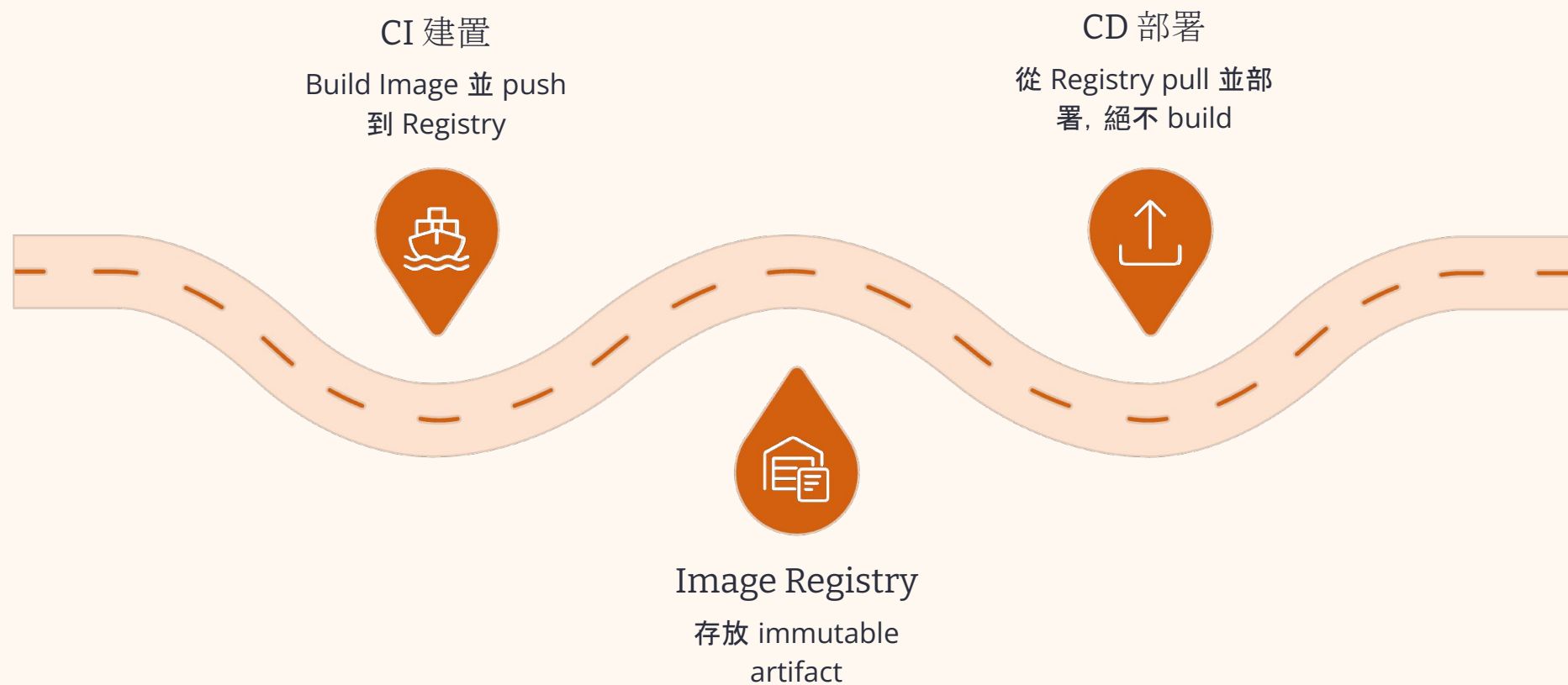
 您會看到 Docker Image 的 Tag 變為 `release-1.0.0`, 同時也會觸發後續的 CD 流程。

 請注意, `act push` 會模擬完整的 GitHub Actions 執行環境。在 Codespaces 中首次執行時, 可能需要下載 Docker 映像檔, 請耐心等待。

! 一個教學妥協要說清楚

有沒有發現 ci.yaml build 過一次 image 了, 可是在 cd.yaml 內又 `docker compose up -d --build` — **重 build 了一次 image**。
前面說 image 是 build once run anywhere。

自相矛盾嗎？是的, 這是為了 lab 簡單做的妥協。



📌 Lab 省略 Registry 是為了不卡認證設定。延伸作業方向之一就是要把這段補上。

設計取舍

為什麼 Lab 用 Docker Compose, 不用 K8s ?

設計取舍

課程目標是**理解 CI/CD pipeline 的本質**，不是學雲端設定。

K8s + cloud registry 設好，可能就吃掉整堂課。

概念相通

Docker Compose 跟 K8s 都是「描述要跑什麼 container」。CD 流程**只差最後一步**——把 `docker compose up` 換成 `kubectl apply` 而已。

- 👉 真實工作：將 Lab 的 `ci.yml` 最後一段替換為『Push Image 到 Registry』；接著透過 ArgoCD 或 `cd.yml` 進行『Pull Image 並執行部署（如 `kubectl apply`）』。這套從自動化建置到環境同步的流程，就是企業級 CD 的核心，其 Pipeline 的骨架與 Lab 是完全一致的。

練習 A

練習 A: 加上 lint 跟 format check

在 `.github/workflows/ci.yml` 的 steps 加:

```
- name: Format check
  run: npm run format:check

- name: Lint
  run: npm run lint
```

驗收:

`act push` 會看到這兩步驟有跑

2. 故意把某個檔案格式弄壞 → CI 在 format:check 失敗
3. 故意違反 lint 規則 → CI 在 lint 失敗

快速參考

Lab 常用指令小抄

開發

```
npm install      # 第一次
npm start        # 跑起來
npm run dev      # 熱重載
```

驗證

```
npm run typecheck # 型別檢查
npm test           # 跑測試
npm run lint       # lint
npm run format:check
```

CI/CD 模擬

```
# 模擬 push 事件(預設跑全部)
act push
# 只跑 ci.yml
act push -W .github/workflows/ci.yml
act pull_request # 模擬 PR 事件
act -j test      # 只跑名為 test 的 job
act -l           # 列出所有 workflow 跟 job
```

常見問題

常見踩雷

Q: Fork 後 Actions 沒跑？

A: 點 Actions 分頁, 啟用 workflow

Q: act 第一次跑很慢？

A: 它要拉 docker image, 正常的, 之後會快

Q: CI 一直紅, 看哪裡？

A: 點失敗的 step, 看 log。同樣的指令在本機重跑(`npm test`)

Q: 不想 push 一直炸 Actions 紀錄？

A: 用 act 在本機先驗

下一步

雲原生這條路還有什麼

CI/CD 之後可以再深入：

主題	做什麼
GitOps + ArgoCD	Pull-based CD, 部署到真 K8s
Helm / Kustomize	管理 K8s manifest 的標準工具
Service Mesh (Istio)	微服務之間的流量治理
Observability	Prometheus + Grafana + Loki
Security Scan	Trivy 掃 image、SBOM
Progressive Delivery	Canary、Blue-Green 部署

 今天打的基礎，是後面所有主題的前提

今天總結

CI/CD 在雲原生扮演的角色

- Container 打包好 → CI 自動 build image
- K8s manifest → CD 自動部署
- Microservices → 每個服務都有獨立 pipeline
- **沒有 CI/CD, 雲原生只是漂亮的架構圖**

今天技能點

- CI = 每次 commit 都驗證, 壞掉立刻知道
- CD = 通過驗證的版本自動部署
- GitHub Actions YAML 怎麼寫
- act 本機模擬, 除錯飛快
- Branch 策略決定 CI/CD 行為
- GitOps = Git 是 cluster 的唯一真相

Q & A / Thank You 🙏

參考資料

GitHub Actions 官方文件 — <https://docs.github.com/actions>

act(本機跑 GitHub Actions) — <https://nektosact.com>

The Twelve-Factor App — <https://12factor.net>

GitOps Principles — <https://opengitops.dev>

ArgoCD — <https://argo-cd.readthedocs.io>

CNCF Cloud Native Definition —

<https://github.com/cncf/toc/blob/main/DEFINITION.md>

Repo



Happy Shipping! 🚀